Type 2: Application Project

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

Autonomous First-Response Crime Scene Exploration Using a Lightweight Onboard Intelligence System

Team Number: [40]

S.No	Register Number	Student Name
1	CB.SC.U4CSE23062	Karthik Raju
2	CB.SC.U4CSE23063	Maheswar Raju
3	CB.SC.U4CSE23212	Vijay Krishna Burra
4	CB.SC.U4CSE23310	Dhinesh Chandra Boddupally
5	CB.SC.U4CSE23421	Kuchipudi Yokshith Sai Ram

Guide: **Mr.Rajeshwar Yadav**

Assistant Professor, Department of CSE

Outline

1. Problem Context
2. Use Case & Deployment Scenario
3. Originality of Use Case
4. Innovation Type
5. Literature Study & Gap Analysis
6. Benchmarking Against Research
7. System Architecture Overview
8. System Architecture
9. Technical Architecture
10. System Workflow
11. Core Technology — Mobile Mamba
12. Mobile Robotic Platform
13. Autonomous Navigation — SLAM
14. ROI Detection Pipeline
15. Data Handling & Security
16. Design Proof
17. Design Validation
18. Standards & Constraints
19. Software Stack
20. Fundamentals Learned
21. Limitations & Mitigation Strategies
22. References

Problem Context

Limitations of Existing Crime Scene Systems

Current systems are not designed for first-response scenarios:

- Depend on **static CCTV infrastructure** with fixed viewpoints
- **Cannot explore or reposition** inside unknown environments
- Focus on **post-event detection accuracy**, not pre-entry safety
- Require **GPU-based models and cloud connectivity**, making them unusable in hostile or disconnected environments

Core Problem

In high-risk situations (e.g., hostage scenarios), first responders must enter unknown indoor environments **without prior spatial awareness**, increasing risk to human life and compromising evidence integrity.

Use Case & Deployment Scenario

Hostage Building — Post-Incident First-Response

Scenario

Hostages are trapped inside a building after a violent incident. Attackers may have **destroyed entry infrastructure**, disabled cameras, and may be using **RF jamming**. **Direct human entry is unsafe without prior situational awareness.**

Mission Flow:

1. Identify **entry point** (door/window/gap)
2. **Explore interior** using SLAM
3. Detect **ROI** (objects, abnormal posture)
4. **Onboard inference** (no cloud)
5. **Store evidence** locally

Deployment Contexts:

- **Law Enforcement:** Pre-entry assessment
- **Forensics:** Scene capture without contamination
- **Emergency:** Unsafe structures
- **Hazardous:** Restricted indoor areas

Originality of Use Case

First-Response Oriented System Design

Proposed System:
An Autonomous First-Response Exploration System for indoor high-risk environments

What existing systems do:

- Focus on **post-event detection** using static cameras
- Assume **fixed infrastructure and connectivity**
- Use **compute-heavy models** not suitable for mobile deployment

What our system introduces:

- **Pre-entry deployment:** Robot enters before human responders
- **Autonomous exploration:** SLAM-based navigation in unknown indoor spaces
- **ROI-based perception:** Focus on situational awareness, not final classification
- **Onboard intelligence:** CPU-only inference (no cloud dependency)
- **Event-driven processing:** Efficient and battery-aware operation

Innovation Type

1. Adaptation

Adapting **autonomous robotic exploration** (commonly used in disaster response and search operations) to the domain of **indoor first-response crime scene assessment**.

- Transition from **large-scale disaster environments** to **confined indoor spaces**
- Low-profile mobile robot designed for **safe navigation in cluttered environments**
- Enables **pre-entry situational awareness** instead of post-event analysis

2. Improvement

Replacing **compute-heavy Transformer / VLM-based models** with:

- **Fine-tuned Mobile Mamba (State-Space Model)**
- **Linear complexity inference** (O(n)) vs quadratic ($O(n^2)$)
- **CPU-only deployment** on embedded hardware (RPi Four(I)

Literature Study & Gap Analysis

Observed in Existing Research

- **Static surveillance:** Fixed-camera anomaly detection systems
- **Heavy models:** Transformer / VLM-based approaches with high compute cost
- **Cloud dependency:** Requires GPU infrastructure and connectivity
- **No mobility:** Lack of integration with autonomous robotic exploration

Identified Research Gaps

- **No unified system:** Missing integration of **SLAM-based navigation + onboard AI + first-response deployment**
- **No lightweight deployment:** Existing models not suitable for **CPU-only embedded systems**
- **Model gap: State-Space Models (Mobile Mamba)** not explored for crime scene ROI detection

Benchmarking Against Research Papers

Comparative Analysis with Existing Systems

Research Work	Core Approach	Key Limitation	Our Edge
LLM-based Video Anomaly Detection (CVPR 2024)	Uses Large Language Models for video reasoning	High computational cost, GPU/cloud dependent	Fully onboard lightweight inference (no cloud dependency)
Text-Prompt Weakly Supervised VAD (CVPR 2024)	Prompt-based anomaly detection from static video	Assumes fixed camera viewpoint	Autonomous SLAM-based exploration with mobile robot
Vision-Language Models for Scene Anomaly (arXiv 2024)	Multimodal scene understanding using VLMs	Computationally expensive, not embeddable	Lightweight State-Space Model (Mobile Mamba) for edge deployment
LLM + Video Interpretation (IEEE Access 2024)	Language-enhanced anomaly reasoning	High latency, unsuitable for real-time robotics	Event-driven inference enabling real-time response
Surveillance & Crime Analysis Survey (2024)	Static infrastructure-based monitoring systems	No mobility, no first-response focus	Autonomous pre-entry exploration for first responders

Key Insight: *Existing systems focus on accuracy, whereas our system integrates mobility, lightweight inference, and real-time deployment for first-response scenarios.*

System Architecture Overview

End-to-End Pipeline

Pipeline Flow:

- **Sense:** Camera, IMU, Depth
- **Navigate:** SLAM + frontier exploration
- **Trigger:** Event-driven activation
- **Perceive:** Mobile Mamba inference
- **Decide:** ROI scoring
- **Store:** Local buffer + SHA-256
- **Sync:** AES-256 cloud backup

Sense $\rightarrow$ *Navigate* $\rightarrow$ *Perceive* $\rightarrow$ *Decide* $\rightarrow$ *Store*

Hardware Platform

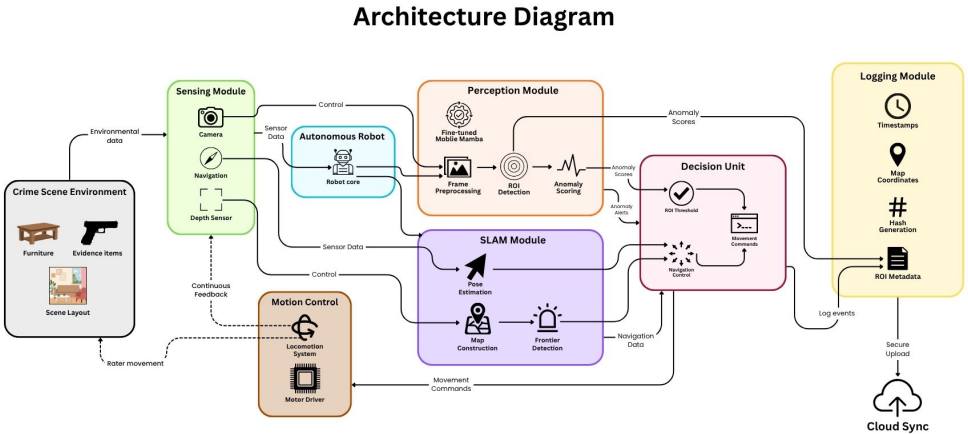
- Mobile robotic platform
- ARM-based processor (Ras Pi)
- Onboard CPU-only inference
- No external dependency

Key Design

- Offline-first operation
- Event-driven processing
- Lightweight deployment

System Architecture

End-to-End Perception–Decision–Action Flow



Technical Architecture

Hardware Platform & Deployment Strategy

Hardware Components

Component	Role
Ras Pi	Onboard CPU processing
Pi Camera Module v2	Visual frame capture
MPU-6050 IMU	Orientation and pose tracking
HC-SR04 Ultrasonic	Obstacle detection
L298N Motor Driver	Locomotion control
Li-Po 3000 mAh	Power supply

Designed for low-cost, embedded, real-world deployment

Connectivity Strategy

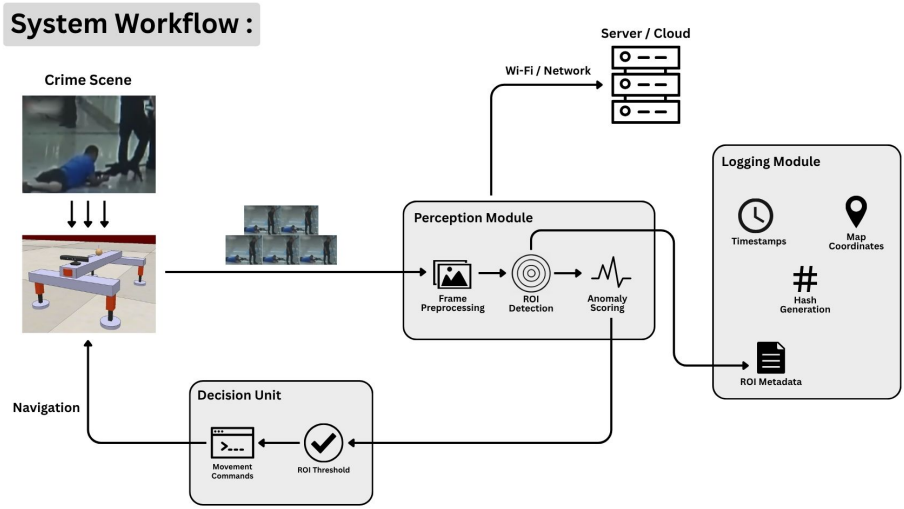
- **Primary:** Fully offline onboard operation
- **Secondary:** Wi-Fi sync when available
- **Jammer scenario:** Data retrieved after mission return

Design Insight

System prioritizes **reliability over connectivity**, ensuring operation in infrastructure-compromised environments.

System Workflow

End-to-End Mission Signal Path



Core Technology: Fine-Tuned Mobile Mamba

Lightweight State-Space Model for Onboard Perception

Fine-Tuning Strategy

- Start from pre-trained Mobile Mamba checkpoint
- Fine-tune on **UCF Crime + curated indoor frames**
- **Layered Fine-Tuning** strategy: freeze backbone early, unfreeze final stage progressively
- Task: **ROI vs Non-ROI classification**
- Optimized for low-resolution indoor scenes

Why Mobile Mamba?

- **Linear complexity** $O(n)$ vs Transformer $O(n^2)$
- Selective State-Space (S6) mechanism
- Low memory footprint
- CPU-friendly real-time inference

Deployment

- Export to ONNX / TFLite
- Runs on embedded CPU
- Target latency: $<$150ms

ROI Detection Pipeline

Event-Driven Perception for Anomaly Detection

Pipeline: *Trigger* $\rightarrow$ *Capture* $\rightarrow$ *Process* $\rightarrow$ *Inference* $\rightarrow$ *Score* $\rightarrow$ *Decision* $\rightarrow$ *Log*

Steps:

1. Sensor trigger (new area)
2. Capture frame (camera)
3. Preprocess (resize, normalise)
4. Run model $\rightarrow$ score
5. Check threshold $\rightarrow$ ROI / not
6. Log data (time, position, score)

Why Event-Driven?

- Less computation
- Saves battery
- Runs only when needed

ROI Types:

- Displaced objects
- Fallen / unusual posture
- Suspicious items (bags)
- Unusual marks or stains

Output: ROI / Non-ROI + confidence score

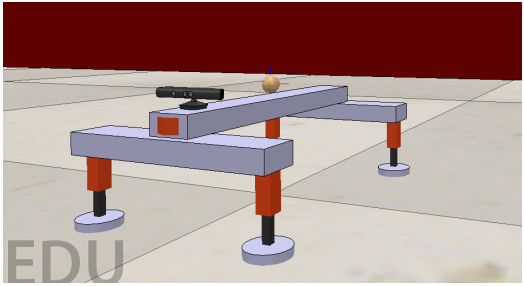
Mobile Robotic Platform

Design Rationale

- **Low-profile chassis:** Navigates confined indoor spaces (under furniture, narrow paths)
- **Stable locomotion:** Handles common indoor surfaces (tile, carpet, light debris)
- **Embedded processing:** ARM-based CPU enables fully onboard computation

Sensing System

- **Camera:** Visual input for ROI detection
- **IMU (Gyro + Accelerometer):** Orientation + motion tracking for navigation
- **Depth sensing:** Obstacle detection and environment mapping



Simulation of autonomous exploration

Watch Simulation Video

Autonomous Navigation — SLAM

Simultaneous Localisation and Mapping for Indoor Exploration

What is SLAM?

Builds a **real-time map** of an unknown environment while **tracking robot position**.

Frontier Exploration:

1. Fuse sensors $\rightarrow$ estimate position
2. Build **map** (free / occupied / unknown)
3. Find **frontiers** (edges of unknown space)
4. Choose nearest frontier as next target
5. Move using safe path
6. Repeat until map is complete or battery $\leq 20\%$

Why Frontier-Based?

- High-coverage mapping
- No redundant exploration
- Complete indoor reconstruction

SLAM Outputs:

- Occupancy grid map
- Robot pose (x, y, θ)
- Obstacle avoidance signals

Data Handling, Security & Communication Strategy

Local Data Handling

- ROI data stored locally
- Non-ROI removed at 80%
- SHA-256 for integrity
- JSON logs (time, position, score)

Communication

- Fully offline operation
- AES-256 sync when Wi-Fi available
- Jammer: data stored until return
- No real-time dependency

Battery & Return

- 3000mAh battery
- Continuous monitoring
- 20% $\rightarrow$ return
- Limitation: recovery not guaranteed

Design Proof

Implementation Evidence & Feasibility Check

Component Readiness

- **Ras Pi:** TFLite $<$150 ms on ARM
- **Mobile Mamba:** Pre-trained +Layered Fine-Tuning
- **SLAM:** GMapping / Hector works on RPi
- **Security:** SHA-256 + AES-256 verified

Dataset Strategy

- UCF-Crime (balanced)
- 50 videos per class (max)
- Split: 80/20
- Balanced sampling (WeightedRandomSampler)

Evaluation

- Metrics: Accuracy, Precision, Recall, F1, latency
- Security: Hash integrity verified
- Testing: Indoor scenes (objects, clutter)

Risk Mitigation

- Latency: INT8 if $>$150 ms
- Battery: 20% return + event-driven mode
- Overfitting: Dropout + staged freezing

Design Validation

Constraint-Driven Design Decisions

Why Mobile Mamba?

- $O(n)$ complexity — runs on CPU (no GPU)
- No full attention $\Rightarrow$ low memory
- TFLite INT8 $<$150 ms on Ras Pi

Why SLAM + Frontier?

- Works without GPS or maps
- Covers all indoor space (no skips)
- Pose (x, y) used in ROI logging

Why Event-Driven?

- Continuous: $\sim$6–7 FPS (not usable)
- Event mode: few inferences per area

Edge Constraints Met

Constraint	Solution
Compute	TFLite + quantized Mobile Mamba on ARM
Power	Li-Po 3000 mAh; 20% return threshold
Storage	JSON + compressed frames; non-ROI removed
Connectivity	Offline-first; no cloud dependency

Validated through trade-offs between accuracy, latency, and deployability

Standards & Constraints

Regulatory Compliance and Resource Budgets

Domain	Requirement	Implementation
Evidence Integrity	Tamper-proof logging	SHA-256 + JSON logs
Data Security	Encrypted storage	AES-256 before sync
Connectivity	Offline-first	No cloud; sync later
Performance	Real-time inference	$<$150 ms (TFLite INT8)
Power	Continuous operation	Li-Po + 20% return
Code Quality	Maintainable code	PEP 8 + modular design

Hardware Constraints

- Ras Pi
- CPU-only inference (no GPU)
- Compact chassis (indoor navigation)

Standards

- NIST SP 800-101
- AES-256 / SHA-256
- ONNX / TFLite

Software Stack

Component Selection & Compatibility

Component	Tech	Purpose
Navigation	GMapping / Hector	Mapping + pose
AI Model	TFLite (Mamba)	ROI detection
Vision	OpenCV + Pi Cam	Frame capture
Sensors	GPIO + I2C	IMU + fusion
Security	hashlib / AES	Hash + encryption
Training	PyTorch	Layered fine-tuning
Language	Python 3	Full system

Platform

- Python + Linux (RPi OS)
- Git + Jupyter (training)

Deployment

- Raspberry Pi OS (64-bit)
- ARM-compatible libraries
- Headless execution

Model Export

PyTorch $\rightarrow$ ONNX $\rightarrow$ TFLite INT8 $\rightarrow$ RPi

Limitations & Mitigation Strategies

Known Constraints and Engineering Responses

High Impact

- **No entry point** Mitigation: Requires at least one accessible opening
- **Robot stuck inside** Mitigation: Retrieval via tether / future swarm approach

Medium Impact

- **Battery depletion** Mitigation: 20% auto-return + event-driven power saving
- **No real-time communication** Mitigation: Data stored locally, synced after mission

Low Impact

- **Limited compute (RPi)** Mitigation: TFLite + quantized Mobile Mamba
- **Single robot coverage** Mitigation: Suitable for small areas; swarm in future

Key Insight

System is designed with **practical constraints in mind**, balancing performance, safety, and deployability.

Fundamentals Learned

Key Concepts Applied in System Design

Autonomous Robotics & SLAM

- Probabilistic localisation and mapping
- Frontier-based exploration
- Real-time pose estimation

Lightweight AI (SSM)

- Mobile Mamba architecture
- Linear-time inference ($O(n)$)
- Layered fine-tuning with differential learning rates

Video Anomaly Detection

- Temporal vs static modelling
- Dataset balancing strategies

Embedded Systems

- ARM-based deployment constraints
- Sensor integration (IMU, camera)
- Edge inference optimisation

Security & Forensics

- SHA-256 for data integrity
- AES-256 encryption
- Structured logging

Project Engineering

- Modular system design
- Architecture modelling

References (1/2)

- A. M. Yildiz *et al.*, “A novel tree pattern-based violence detection model using audio signals,” *Expert Systems with Applications*, vol. 224, p. 120031, 2023.
- H. Chen *et al.*, “UCVL: A benchmark for crime surveillance video analysis with large models,” in *Proc. 5th Int. Conf. on Artificial Intelligence and Industrial Technology Applications (AIITA)*, 2025, pp. 2051–2057.
- C. Liu *et al.*, “Improving anomaly scene recognition with large vision-language models,” 2024.
- Z. Yang, J. Liu, and P. Wu, “Text prompt with normality guidance for weakly supervised video anomaly detection,” in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- H. He *et al.*, “MobileMamba: Lightweight Multi-Receptive Visual Mamba Network,” arXiv preprint arXiv:2411.15941, 2024.
- P. Wu *et al.*, “VadCLIP: Adapting vision-language models for weakly supervised video anomaly detection,” in *Proc. AAAI Conf. on Artificial Intelligence*, 2024, pp. 6074–6082.

References (2/2)



S. Sun *et al.*, “TDSD: Text-driven scene-decoupled weakly supervised video anomaly detection,” in *Proc. ACM Int. Conf. on Multimedia (MM)*, 2024.



L. Zanella *et al.*, “Harnessing large language models for training-free video anomaly detection,” in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2024.



S. Zou *et al.*, “Unlocking vision-language models for video anomaly detection via fine-grained prompting,” arXiv preprint arXiv:2510.02155, 2025.

Thank You